Statistical and Machine Learning

Neural Networks for Images

Katarzyna Reluga

Neural Networks for Images

- Introduction
- Foundations of CNN
  - Convolutional layers
  - Pooling and normalisation
- Common architectures for image classification
- Solving other discriminative vision tasks with CNNs
- Generating images by inverting CNNs

Neural Networks for Images

- **Introduction**
- Foundations of CNN
  - Convolutional layers
  - Pooling and normalisation
- Common architectures for image classification
- Solving other discriminative vision tasks with CNNs
- Generating images by inverting CNNs

Recap: multilayer perceptrons (MLPs)

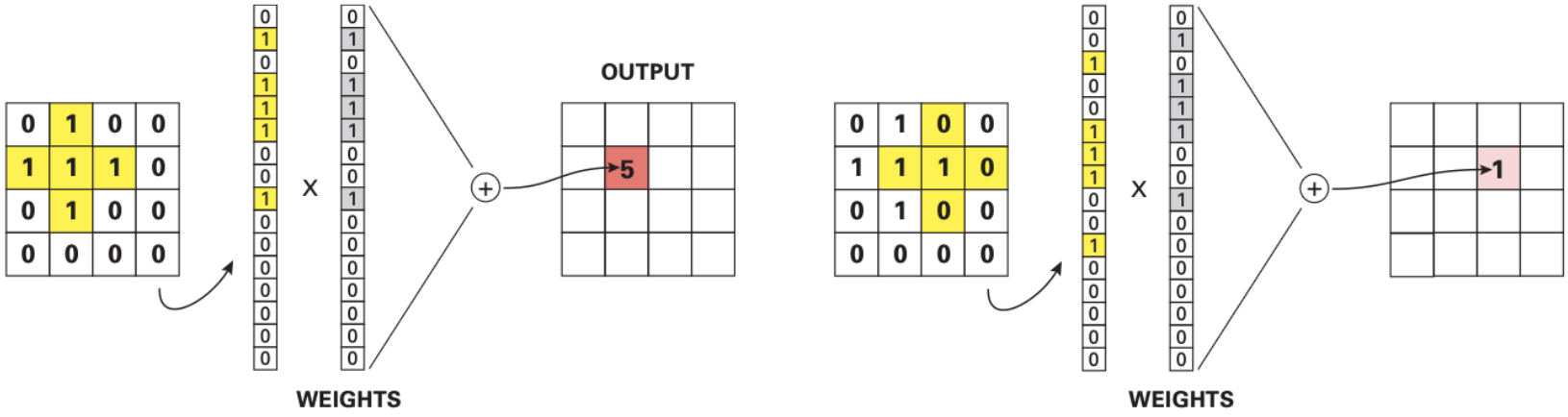
- An MLP hidden layer computes activations $\mathbf{z} = \varphi(\mathbf{W}\mathbf{x})$, where $\mathbf{x}$ is the input, $\mathbf{W}$ the weights, $\varphi(\cdot)$ a nonlinearity
- The j'th hidden unit: $z_j = \varphi(\mathbf{w}_j^\top \mathbf{x})$
- Interpretation: an inner product compares the input $\mathbf{x}$ to a learned template/pattern $\mathbf{w}_j$
- If the match is good (large positive inner product), the activation is large (with a ReLU), signalling that pattern j is present in the input

Why not apply MLPs directly to images?

Image input: $\mathbf{x} \in \mathbb{R}^{WHC}$, where W, H are width/height and C is the number of channels (e.g. $C = 3$ for RGB). Three problems with a plain MLP layer on such inputs:

- We would need a **different-sized** weight matrix $\mathbf{W}$ for every input image size
- For a fixed-size input, the num. of parameters is prohibitive: $\mathbf{W}$ has size $(W \times H \times C) \times D$, with D the num. of hidden units
- A pattern occurring in one location may not be recognized in another, since weights are **not shared** across locations – i.e. the model lacks **translation invariance**

Illustration of the lack of translation invariance

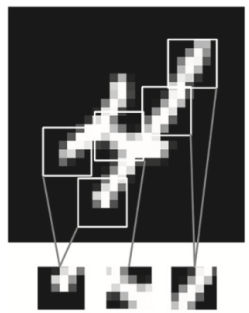


- MLPs is not translation invariant – detecting patterns in 2d images does not work.
- Consider a weight vector to act as a matched filter for detecting the desired cross-shape.
- This will give a strong response of 5 if the object is on the left, but a weak response of 1 if the object is shifted over to the right.

From templates to convolutions

To fix these problems use **convolutional neural networks (CNNs)**, which replace matrix multiplication with a **convolution** operation:

- Divide the input into overlapping 2d **image patches**
- Compare each patch against a set of small weight matrices, or **filters**, each representing some part of an object – a form of **template matching**
- Filters are learned from data and are small (e.g. 3×3 or 5×5) $\Rightarrow$ far fewer parameters
- Template matching by convolution, not by product – the resulting model is translationally invariant – useful when we only care **whether** an object is present, not **where**



Neural Networks for Images

- Introduction
- **Foundations of CNN**
  - **Convolutional layers**
  - Pooling and normalisation
- Common architectures for image classification
- Solving other discriminative vision tasks with CNNs
- Generating images by inverting CNNs

Convolution: general definition

The **convolution** of two functions $f, g : \mathbb{R}^D \to \mathbb{R}$ is

$$[f \circledast g](\mathbf{z}) = \int_{\mathbb{R}^D} f(\mathbf{u})\, g(\mathbf{z} - \mathbf{u})\, d\mathbf{u}$$

- Replace the functions with finite-length vectors (functions evaluated on a finite grid of points)
- Evaluating f on $\{-L, \ldots, L\}$ gives the **weight vector** (also called a **filter** or **kernel**) $w_{-L} = f(-L), \ldots, w_L = f(L)$
- Evaluating g on $\{-N, \ldots, N\}$ gives the **feature vector** $x_{-N} = g(-N), \ldots, x_N = g(N)$

Convolution in 1d

With $\mathbf{w}$ indexed on $\{-L, \ldots, L\}$ and $\mathbf{x}$ indexed on $\{-N, \ldots, N\}$, convolution becomes

$$[\mathbf{w} \circledast \mathbf{x}](i) = w_{-L}x_{i+L} + \cdots + w_{-1}x_{i+1} + w_0 x_i + w_1 x_{i-1} + \cdots + w_L x_{i-L}$$

- We "flip" the weight vector $\mathbf{w} = [5, 6, 7]$ (its indices are reversed), then "drag" it over $\mathbf{x} = [1, 2, 3, 4]$, summing the local window at each point to yield $z = [5, 16, 34, 52, 45, 28]$

-	-	1	2	3	4	-	-	
7	6	5	-	-	-	-	-	$z_0 = x_0 w_0 = 5$
-	7	6	5	-	-	-	-	$z_1 = x_0 w_1 + x_1 w_0 = 16$
-	-	7	6	5	-	-	-	$z_2 = x_0 w_2 + x_1 w_1 + x_2 w_0 = 34$
-	-	-	7	6	5	-	-	$z_3 = x_1 w_2 + x_2 w_1 + x_3 w_0 = 52$
-	-	-	-	7	6	5	-	$z_4 = x_2 w_2 + x_3 w_1 = 45$
-	-	-	-	-	7	6	5	$z_5 = x_3 w_2 = 28$

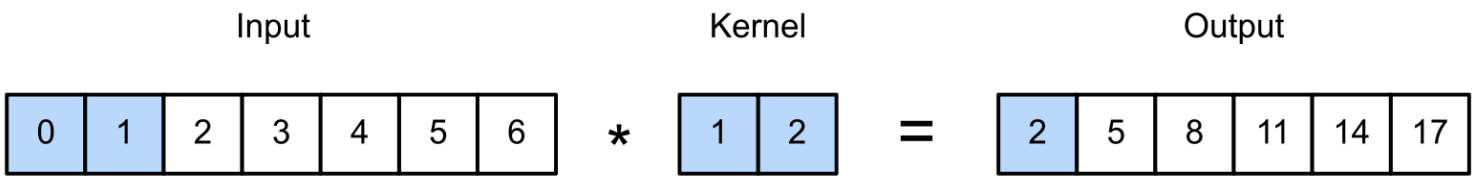
Cross-correlation

A closely related operation does **not** flip $\mathbf{w}$ first:

$$[\mathbf{w} * \mathbf{x}](i) = w_{-L}x_{i-L} + \cdots + w_{-1}x_{i-1} + w_0 x_i + w_1 x_{i+1} + \cdots + w_L x_{i+L}$$

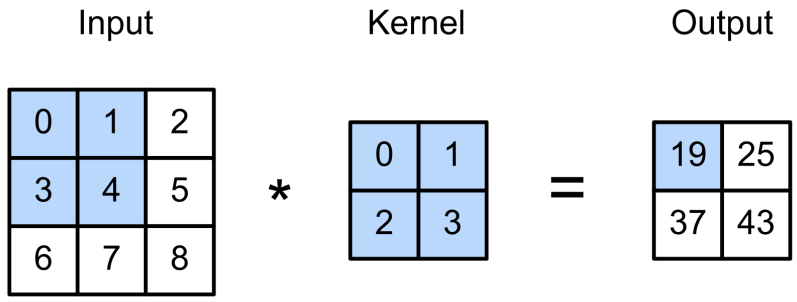
- This is called **cross-correlation**
- If the weight vector is symmetric, convolution and cross-correlation coincide
- **Deep learning convention:** "convolution" usually *means* cross-correlation – we follow this convention

Using non-negative indices $\{0, \ldots, L-1\}$ for $\mathbf{w}$ and $\{0, \ldots, N-1\}$ for $\mathbf{x}$ (no negative indices): $[\mathbf{w} \circledast \mathbf{x}](i) = \sum_{u=0}^{L-1} w_u x_{i+u}$



Convolution in 2d

The 2d cross-correlation with filter $\mathbf{W}$ of size $H \times W$ is:
$[\mathbf{W} \circledast \mathbf{X}](i,j) = \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} w_{u,v}\, x_{i+u,j+v}$



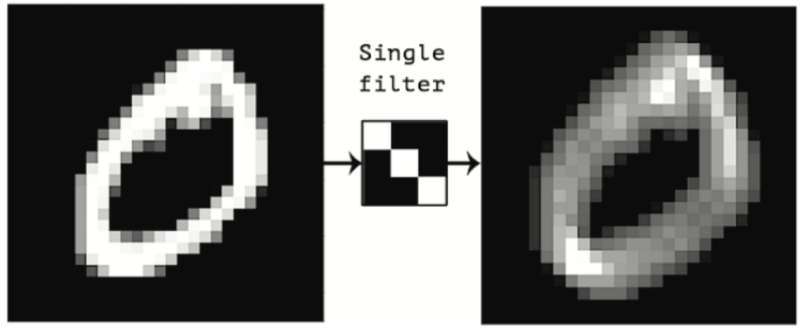
Worked example: convolving a 3×3 input $\mathbf{X}$ with a 2×2 kernel $\mathbf{W}$ gives a 2×2 output $\mathbf{Y}$:

$$\mathbf{Y} = \begin{pmatrix} w_1 & w_2 \\ w_3 & w_4 \end{pmatrix} \circledast \begin{pmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{pmatrix}$$

$$= \begin{pmatrix} (w_1 x_1 + w_2 x_2 + w_3 x_4 + w_4 x_5) & (w_1 x_2 + w_2 x_3 + w_3 x_5 + w_4 x_6) \\ (w_1 x_4 + w_2 x_5 + w_3 x_7 + w_4 x_8) & (w_1 x_5 + w_2 x_6 + w_3 x_8 + w_4 x_9) \end{pmatrix}$$

2d convolution as template matching

- Output at (i, j) is large whenever the image patch centered on (i, j) is similar to the filter $\mathbf{W}$
- If $\mathbf{W}$ encodes an oriented edge, convolving with it makes the output **heat map** "light up" wherever the image contains an edge of that orientation
- More generally, convolution implements **feature detection**: the output $\mathbf{Y} = \mathbf{W} \circledast \mathbf{X}$ is called a **feature map**



Example: Convolving a 2d image (left) with a 3 $\times$3 filter (middle) produces a 2d response map (right). The bright spots of the response map correspond to locations in the image which contain diagonal lines sloping down and to the right.

Convolution as matrix–vector multiplication

Convolution is a linear operator, so it can be written as a matrix multiplication. Flatten $\mathbf{X}$ into a vector $\mathbf{x}$, and multiply by a **Toeplitz-like matrix $\mathbf{C}$** built from the kernel $\mathbf{W}$:

$$\mathbf{y} = \mathbf{C}\mathbf{x} = \left(\begin{array}{ccc|ccc|ccc} w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 \\ 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 \\ 0 & 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 \end{array}\right) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \end{pmatrix}$$

$$= \begin{pmatrix} w_1 x_1 + w_2 x_2 + w_3 x_4 + w_4 x_5 \\ w_1 x_2 + w_2 x_3 + w_3 x_5 + w_4 x_6 \\ w_1 x_4 + w_2 x_5 + w_3 x_7 + w_4 x_8 \\ w_1 x_5 + w_2 x_6 + w_3 x_8 + w_4 x_9 \end{pmatrix}$$

- The 2×2 output is recovered by reshaping the 4×1 vector $\mathbf{y}$ back into $\mathbf{Y}$

CNNs vs. MLPs

CNNs are MLPs whose weight matrices have a special structure:

- **Sparse**: most entries are zero
- **Tied**: the nonzero elements are shared across spatial locations (the same $w_1, \ldots, w_4$ reappear everywhere)

This structure implements **translation invariance**, and massively reduces the number of parameters compared to a standard fully-connected (dense) layer.

Boundary conditions, padding and strided convolution

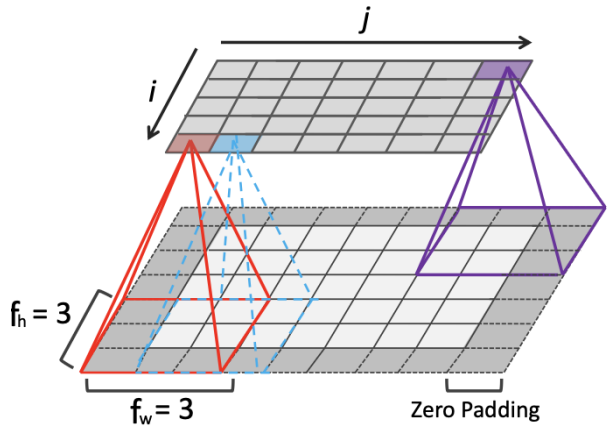
Padding

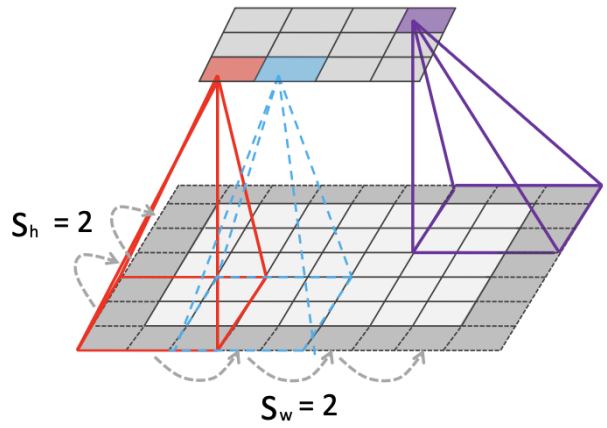
- Convolving an $f_h \times f_w$ filter over an $x_h \times x_w$ image (no padding) gives an output of size $(x_h - f_h + 1) \times (x_w - f_w + 1)$ – called **valid convolution** (the filter never "slides off the ends")
- To keep the output the same size as the input, pad the image with a border of zeros: **zero-padding**, giving **same convolution**
- With padding p_h, p_w on each side, the output size is: $(x_h + 2p_h - f_h + 1) \times (x_w + 2p_w - f_w + 1)$

Strided convolution

- Neighboring outputs overlap heavily $\Rightarrow$ redundant. We can skip every s'th input position: **strided convolution**
- With padding p_h, p_w and strides s_h, s_w, the output size is: $\left\lfloor \frac{x_h+2p_h-f_h+s_h}{s_h} \right\rfloor \times \left\lfloor \frac{x_w+2p_w-f_w+s_w}{s_w} \right\rfloor$

Example





Example: 5×7 image, 3×3 filter, $p = 1$, $s = 2$.

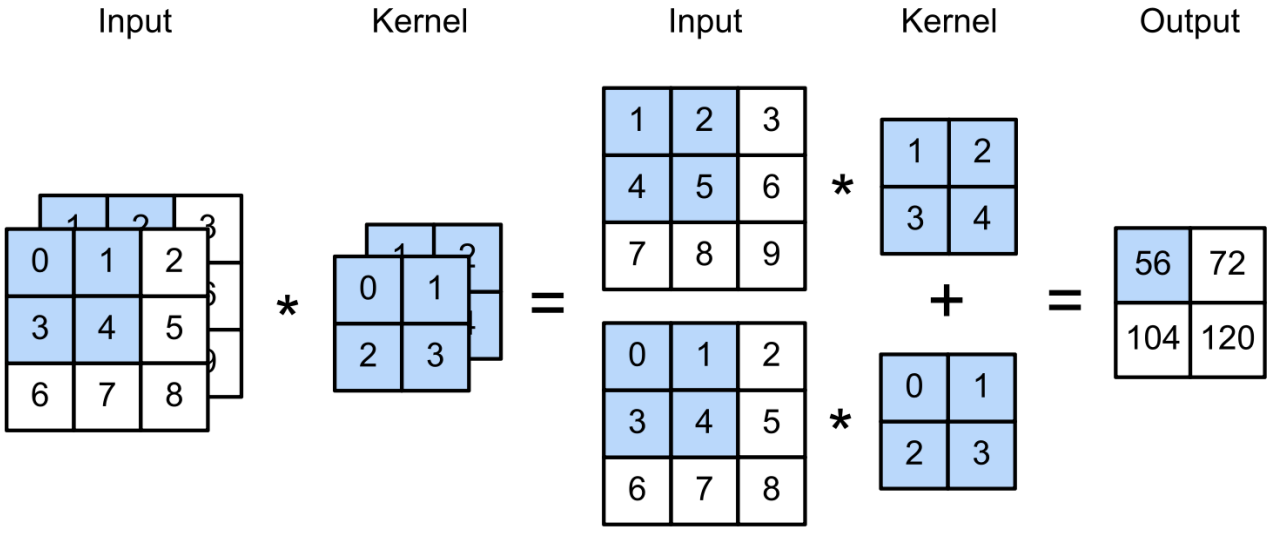
a) Padding: $(5 + 2 - 3 + 1) \times (7 + 2 - 3 + 1) = 5 \times 7$, i.e., if $2p = f - 1$, the output has the same size as the input.

b) Padding and striding: $\left\lfloor \frac{5+2-3+2}{2} \right\rfloor \times \left\lfloor \frac{7+2-3+2}{2} \right\rfloor = 3 \times 4$ – output smaller than the input.

Multiple input channels

- Real images have multiple input **channels** C (e.g. RGB, or hyperspectral bands)
- Define one kernel per input channel $\Rightarrow$ $\mathbf{W}$ becomes a 3d tensor
- Convolve channel c of the input with kernel $\mathbf{W}_{:,:,c}$, then sum over channels (with stride s and bias b):

$$z_{i,j} = b + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} x_{si+u,\, sj+v,\, c}\, w_{u,v,c}$$



Multiple output channels

- A single weight matrix detects only one kind of feature; we usually want to detect many kinds at once
- Make $\mathbf{W}$ a 4d tensor: the filter for feature type d in input channel c is stored in $\mathbf{W}_{:,:,c,d}$

$$z_{i,j,d} = b_d + \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} \sum_{c=0}^{C-1} x_{si+u,\, sj+v,\, c}\, w_{u,v,c,d}$$

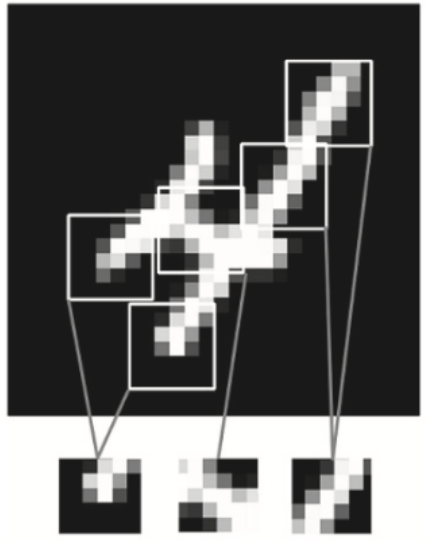
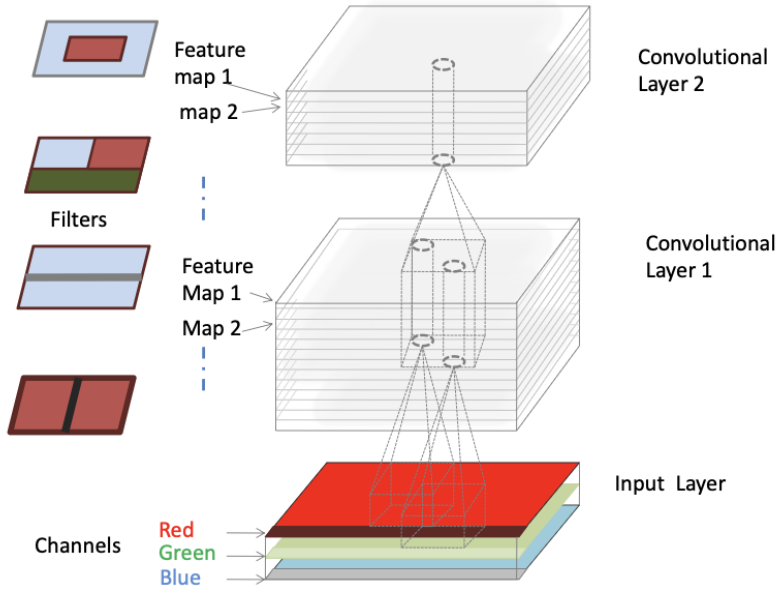


Illustration: multiple output channels



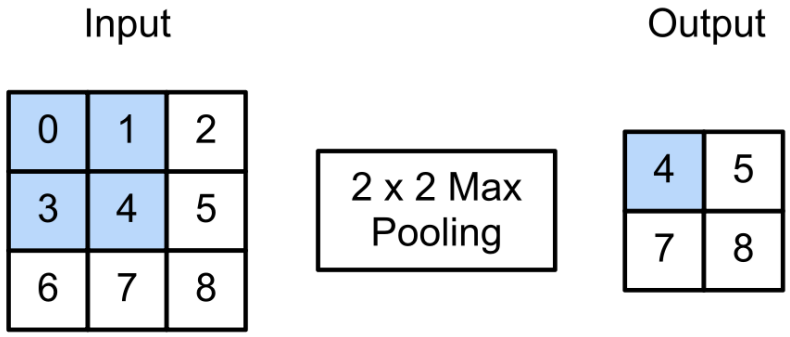
Each vertical cylindrical column denotes the set of output features at a given location $\mathbf{z}_{i,j,1:D}$ (so called **hypercolumn**) – each element is a different weighted combination of the C features in the receptive field of each of the feature maps in the layer below.

Neural Networks for Images

- Introduction
- **Foundations of CNN**
  - Convolutional layers
  - **Pooling and normalisation**
- Common architectures for image classification
- Solving other discriminative vision tasks with CNNs
- Generating images by inverting CNNs

Pooling layers

- Convolution preserves the location of input features $\rightarrow$ **equivariance**
- Sometimes we instead want invariance to location — e.g. for image classification we may just want to know **whether** an object (a face, say) is present anywhere
- **Max pooling**: take the maximum over a local window of incoming values



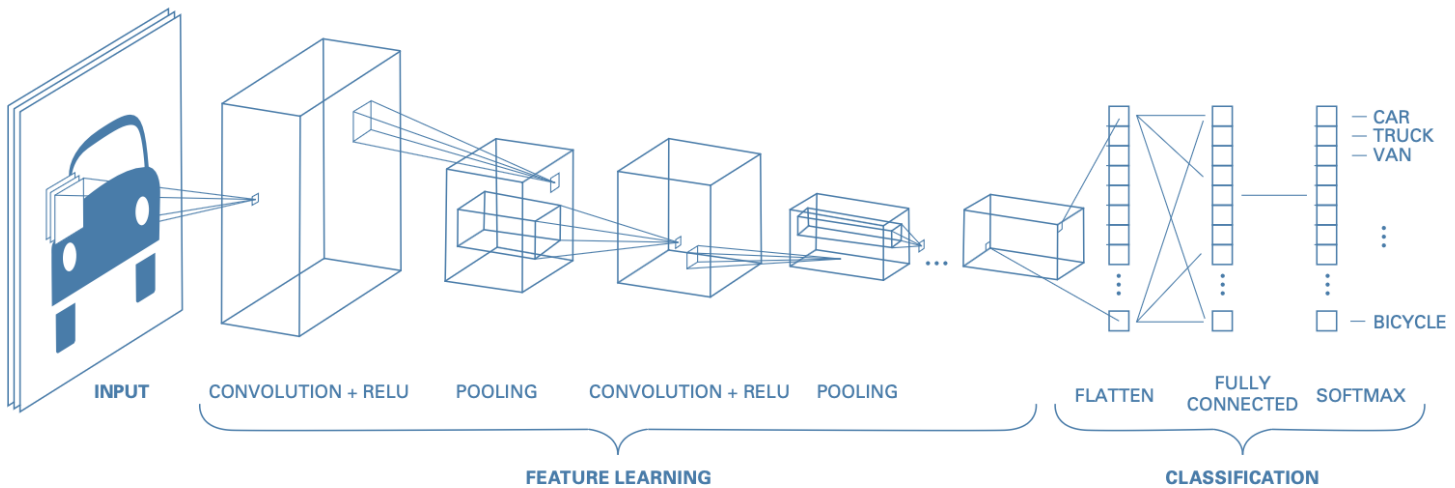
- **Average pooling**: replace the max with the mean
- Pooling is applied to each feature channel independently
- Either way, the output neuron responds similarly no matter *where* in its receptive field the input pattern occurs

Global average pooling

- Averaging over **all** spatial locations of a feature map is called **global average pooling**
- Converts an $H \times W \times D$ feature map into a $1 \times 1 \times D$ map, reshaped to a D-dimensional vector
- That vector is passed to a fully connected layer, mapping it to a C-dimensional vector, then to a softmax
- Because the final feature map is always summarized into a fixed D-dimensional vector, the classifier can be applied to images of **any size**

A basic CNN architecture

- Common design pattern: alternate convolutional layers with max-pooling layers, ending in a final linear classification layer



Normalization layers: motivation

- The basic convolution + pooling design works well for shallow CNNs
- Scaling to **deep** CNNs is harder, due to vanishing/exploding gradients
- A common fix: add extra layers that standardize the statistics of the hidden units (zero mean, unit variance) — just as we standardize the inputs to many models

Batch normalization (BN)

The most popular normalization layer. Standardizes the distribution of activations within a layer, averaged across the examples in a minibatch $\mathcal{B}$, i.e., replace the activation vector $\mathbf{z}_n$ with $\tilde{\mathbf{z}}_n$ computed as follows:

$$\tilde{\mathbf{z}}_n = \boldsymbol{\gamma} \odot \hat{\mathbf{z}}_n + \boldsymbol{\beta} \qquad \hat{\mathbf{z}}_n = \frac{\mathbf{z}_n - \boldsymbol{\mu}_{\mathcal{B}}}{\sqrt{\boldsymbol{\sigma}^2_{\mathcal{B}} + \epsilon}}$$

$$\boldsymbol{\mu}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{z} \in \mathcal{B}} \mathbf{z} \qquad \boldsymbol{\sigma}^2_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{\mathbf{z} \in \mathcal{B}} (\mathbf{z} - \boldsymbol{\mu}_{\mathcal{B}})^2$$

- $\boldsymbol{\beta}, \boldsymbol{\gamma}$: learnable per-layer parameters; $\epsilon > 0$: small constant for numerical stability
- The transformation is differentiable, so gradients flow back to the layer input and to $\boldsymbol{\beta}, \boldsymbol{\gamma}$

Batch normalization: training vs. test time

- At the **input** layer, BN reduces to the usual standardization (mean/variance computed once, since data is static)
- For **internal** layers, the empirical mean/variance keep changing as parameters adapt – "internal covariate shift" – so $\boldsymbol{\mu}, \boldsymbol{\sigma}^2$ must be recomputed every minibatch during training
- At **test time** we may see a single input, so batch statistics can't be computed:
  - Standard fix: after training, compute $\boldsymbol{\mu}_l, \boldsymbol{\sigma}_l^2$ for layer l over the *whole* training set, then freeze them alongside $\boldsymbol{\beta}_l, \boldsymbol{\gamma}_l$
  - **Fused batchnorm**: fold the frozen BN into the preceding linear layer. If that layer computes $\mathbf{XW} + \mathbf{b}$, the combined layer is $\mathbf{XW}' + \mathbf{b}'$ with $\mathbf{W}' = \boldsymbol{\gamma} \odot \mathbf{W}/\boldsymbol{\sigma}$ and $\mathbf{b}' = \boldsymbol{\gamma} \odot (\mathbf{b} - \boldsymbol{\mu})/\boldsymbol{\sigma} + \boldsymbol{\beta}$

Why does batch norm help?

The benefits to training speed and stability can be dramatic, especially for deep CNNs – the exact reasons remain debated:

- Makes the optimization landscape significantly smoother
- Reduces sensitivity to the learning rate
- Acts like a regularizer; can be cast as a form of approximate Bayesian inference
- Downside: reliance on minibatch statistics gives unstable parameter estimates with small batch sizes (partially addressed by **batch renormalization**)

Other kinds of normalization layer

All of these standardize $\hat{z}_i = (z_i - \mu_i)/\sigma_i$ over some set $\mathcal{S}_i$ of tensor locations (indexed by batch i_N, height i_H, width i_W, channel i_C), then rescale $\tilde{z}_i = \gamma_c \hat{z}_i + \beta_c$. They differ only in how $\mathcal{S}_i$ is chosen:

- **Batch norm**: pool over batch, height, width; match on channel
- **Layer normalization**: pool over channel, height, width; match on batch (a single group containing all channels)
- **Instance normalization**: separate statistics per example *and* per channel (C groups)
- **Group normalization**: generalizes both — pool within groups of channels (between 1 and C channels per group); often improves training speed and accuracy over layer/instance norm
- **Filter response normalization**: per (channel, batch) group as in instance norm, but divides only by the mean squared norm (no mean-centering); paired with a **thresholded linear unit** $y = \max(x, \tau)$ (τ learnable) to stop activations drifting away from 0 – works well even with batch size 1
- **Normalizer-free networks** : train deep residual networks with **no normalization layer at all**, replacing it with adaptive gradient clipping – faster and more accurate than competitive batchnorm baselines

Neural Networks for Images

- Introduction
- Foundations of CNN
  - Convolutional layers
  - Pooling and normalisation
- **Common architectures for image classification**
- Solving other discriminative vision tasks with CNNs
- Generating images by inverting CNNs

Image classification: setup

- **Image classification**: estimate a function $f : \mathbb{R}^{H \times W \times K} \rightarrow \{0, 1\}^C$
  - K: number of input channels (e.g. $K = 3$ for RGB images)
  - C: number of class labels
- This section briefly reviews CNN architectures developed over the years to tackle this task
- See this website for an up-to-date repository of code and models (PyTorch)

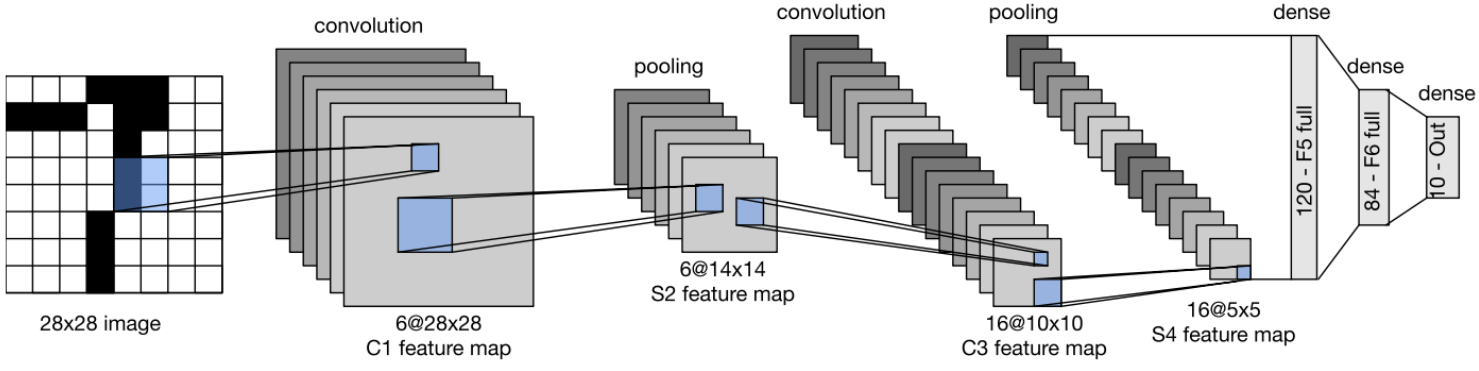
LeNet (1998): Early CNN for Digit Recognition

Core idea

- Early CNN by Yann LeCun et al., trained on MNIST digits
- Reached **98.8% test accuracy after 1 epoch**, vs. **95.9%** for an MLP
- Further training approached the limit imposed by label noise

Limitation and extension

- Real handwriting often appears as **strings**, not isolated digits
- Requires **segmentation + classification**
- LeCun et al. combined CNNs with a CRF-like sequence model
- System was deployed by the **US Postal Service**



AlexNet (2012)

Impact

- AlexNet, by Alex Krizhevsky, marked the ImageNet breakthrough
- Reduced ImageNet **top-5 error** from **26%** to **15%**

Key differences from LeNet

- **Deeper**: 8 trainable layers vs. 5
- Used **ReLU** instead of tanh
- Used **dropout** for regularization
- Stacked multiple convolutional layers before pooling

Scale and training

- About **60M parameters**, mostly in the fully connected layers
- Trained across **two GPUs** due to memory limits

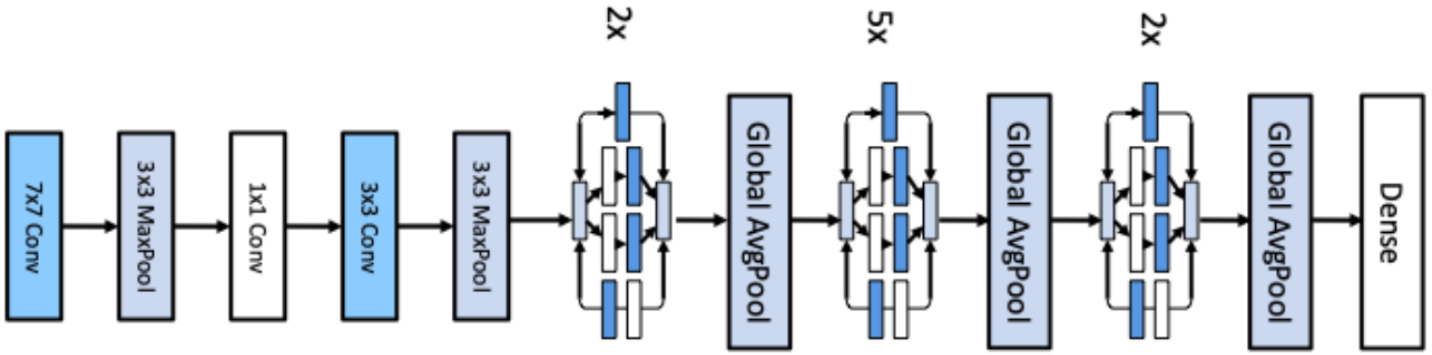
GoogLeNet / Inception

Core idea

- Developed by Google; name combines “Google” and “LeNet”
- Introduced the **inception block**: parallel convolutional paths with different filter sizes
- Lets the network combine features at multiple spatial scales

Architecture and legacy

- Built from **9 inception blocks**, followed by global average pooling



ResNet (2015)

Core idea

- ImageNet 2015 winner from Microsoft
- Introduced **residual blocks** with skip connections:$\mathbf{x}_{l+1} = \varphi(\mathbf{x}_l + \mathcal{F}_l(\mathbf{x}_l))$
- The block learns the **residual** rather than the full mapping

Why it works

- Skip connections let gradients flow directly to earlier layers
- Enables training of much deeper CNNs
- Use padding or 1×1 convolutions when dimensions/channels need matching

Extensions of ResNet

ResNet-18

- Practical shallow variant of the 152-layer ImageNet ResNet
- 18 trainable layers: initial 7×7 conv, 8 residual blocks with two 3×3 convs each, final FC layer
- Downsamples by $2^5 = 32$: $224 \times 224 \to 7 \times 7$ before global average pooling

PreResNet and Wide ResNet

- PreResNet moves activation before the residual addition, enabling very deep networks: $\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}_l(\varphi(\mathbf{x}_l))$
- This preserves a cleaner identity path than $\mathbf{x}_{l+1} = \varphi(\mathbf{x}_l + \mathcal{F}(\mathbf{x}_l))$
- Wide ResNet improves performance by increasing channel width instead of depth

DenseNet

Core idea

- ResNet **adds** residual outputs; DenseNet **concatenates** them
- Each layer receives the feature maps from **all previous layers**:

$$\mathbf{x}_{l+1} = [\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_l]$$

- This makes earlier features directly reusable throughout the network

Trade-off

- Dense connectivity improves feature reuse and gradient flow
- But concatenation increases channel count and computation
- Use 1×1 convolutions and pooling to control model size

Neural Architecture Search (NAS)

Core idea

- CNN design often reuses the same building blocks: convolutions, pooling, skips, strides,
- **NAS** automates this design process by optimizing architectures for validation performance
- Related broader field: **AutoML**; NAS focuses on neural network architectures
- NAS might optimize: accuracy, model size, training cost, and inference speed, example: **EfficientNetV2**
- Modern CNNs such as **ConvNeXt** also show how incremental architectural choices can yield strong results

Main challenge

- Each architecture evaluation is expensive because it may require training a model
- Common shortcuts: Bayesian optimization, differentiable search, or training-free predictors such as neural tangent kernel (NTK)-based methods

Neural Networks for Images

- Introduction
- Foundations of CNN
  - Convolutional layers
  - Pooling and normalisation
- Common architectures for image classification
- **Solving other discriminative vision tasks with CNNs**
- Generating images by inverting CNNs

Beyond image classification

- Image classification is just one vision task; many others can also be tackled with CNNs
- Each new task tends to introduce a new architectural innovation, adding to our library of building blocks
- We cover following tasks in the rest of the lecture:
  - Image tagging
  - Object detection
  - Instance segmentation
  - Semantic segmentation
  - Human pose estimation

Image tagging: from one label to many labels

- **Image classification** usually assumes a **single label** per image.
- This means the classes are treated as mutually exclusive: $\boldsymbol{y} \in \{1, \ldots, C\}$
- Example: image $\longmapsto$ dog.
- However, many images contain several relevant objects or concepts. For example, the same image may contain: $\{\text{dog}, \text{person}, \text{grass}, \text{outdoor}\}$. In this case, it is unclear what **the** single correct label should be.
- **Image tagging** is therefore a **multi-label prediction** problem: we predict all relevant tags, not just one class.
- The output space is $\mathcal{Y} = \{0, 1\}^C$ where C is the number of possible tag types.
- Each component indicates whether a tag is present:

$$y_c = \begin{cases} 1, & \text{if tag } c \text{ is present,} \\ 0, & \text{otherwise.} \end{cases}$$

Image tagging: independent logistic outputs

- In single-label classification, the final layer often uses a **softmax**: $\hat{y}_c = \frac{\exp(z_c)}{\sum_{k=1}^{C} \exp(z_k)}$, $c = 1, \ldots, C$.
- Softmax creates a probability distribution over mutually exclusive classes: $\sum_{c=1}^{C} \hat{y}_c = 1$.
- This is not suitable for image tagging, because several tags may be correct at the same time.
- For multi-label prediction, we replace the final softmax with C independent **logistic units**: $\hat{y}_c = \sigma(z_c) = \frac{1}{1+\exp(-z_c)}$.
- Each output is interpreted as the probability that tag c is present: $\hat{y}_c \approx p(y_c = 1 \mid \boldsymbol{x})$.
- The predicted tag set is obtained by thresholding: $\hat{\mathcal{Y}} = \{c : \hat{y}_c > \tau\}$, use, fro example, $\tau = 0.5$.
- Tagging datasets can be built from social media hashtags.
- These labels are often noisy: some tags are used sparsely, others are subjective, or not visually well-defined.
- Even noisy tags can still be useful for **pre-training**, because they provide large amounts of weak supervision.

Object detection: anchor boxes

- **Object detection**: return a set of **bounding boxes** locating objects of interest, together with their class labels.
  - Example: detect all dogs, cats, and cars in an image.
  - **Face detection** is a special case with only one class of interest.
- A simple approach is to turn object detection into a **closed world** problem.
- Instead of searching over all possible subimages (boxes), we define a finite set of candidate boxes, called **anchor boxes**.
- Anchor boxes are placed at different: spatial locations, scales, aspect ratios.
- For each anchor box, the network predicts: a) whether the box contains an object, b) which object class it contains, c) how the box should be shifted or resized to better match the true object.
- The number of classes C is fixed by the dataset or task.
  - If we detect cats, dogs, and cars, then $C = 3$.
  - If we only detect faces, then $C = 1$.

Object detection: predictions per anchor box

- Mathematically, we learn a function

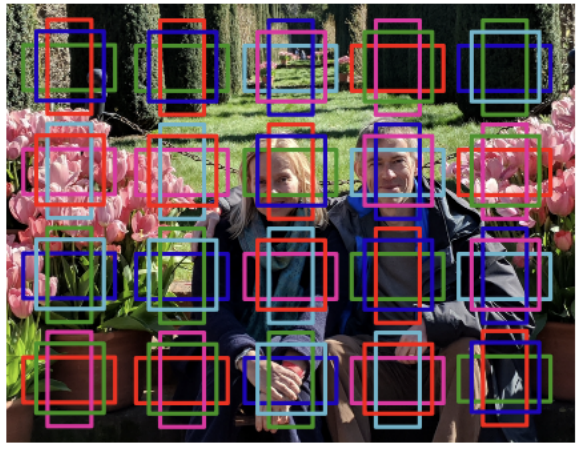
$$f_{\boldsymbol{\theta}} : \mathbb{R}^{H \times W \times K} \to [0,1]^{A \times A} \times \{1, \ldots, C\}^{A \times A} \times (\mathbb{R}^4)^{A \times A}$$

  where H, W: height and width of the input image, K: num. of input channels, A: num. of anchor-box locations per dimension, C: num. of object classes.
- For each anchor-box location (i, j), the network predicts:
  - **objectness score** $p_{ij} \in [0, 1]$, i.e. whether there is an object in this anchor box,
  - **class label** $y_{ij} \in \{1, \ldots, C\}$ e.g. cat, dog, car,
  - **box offset** $\boldsymbol{\delta}_{ij} \in \mathbb{R}^4$, i.e. how to correct the anchor box.
- The offset is a continuous-valued regression target, for example $\boldsymbol{\delta}_{ij} = (\Delta x, \Delta y, \Delta w, \Delta h)$, where $\Delta x, \Delta y$ shift the box center and $\Delta w, \Delta h$ adjust its size.
- Therefore: final predicted box = anchor box + predicted offset.
- This allows **sub-grid spatial localization**: even though anchors lie on a fixed grid, the predicted box can move continuously and locate the object more precisely than the grid resolution.

Object detection: models & illustration

- Examples of specific implementations: **single shot detector**, **YOLO** – "you only look once", etc.

(a)



(b)

(a) Illustration of face detection (photo of author and his wife Margaret. Image processed by Jonathan Huang using SSD face model.)

(b) Illustration of anchor boxes

Instance vs. Semantic Segmentation

Object detection

- **Object detection** predicts a class label and a bounding **box** for each detected object, e.g., car + bounding box.
- The box localizes the object approximately, but it does not describe its exact shape.

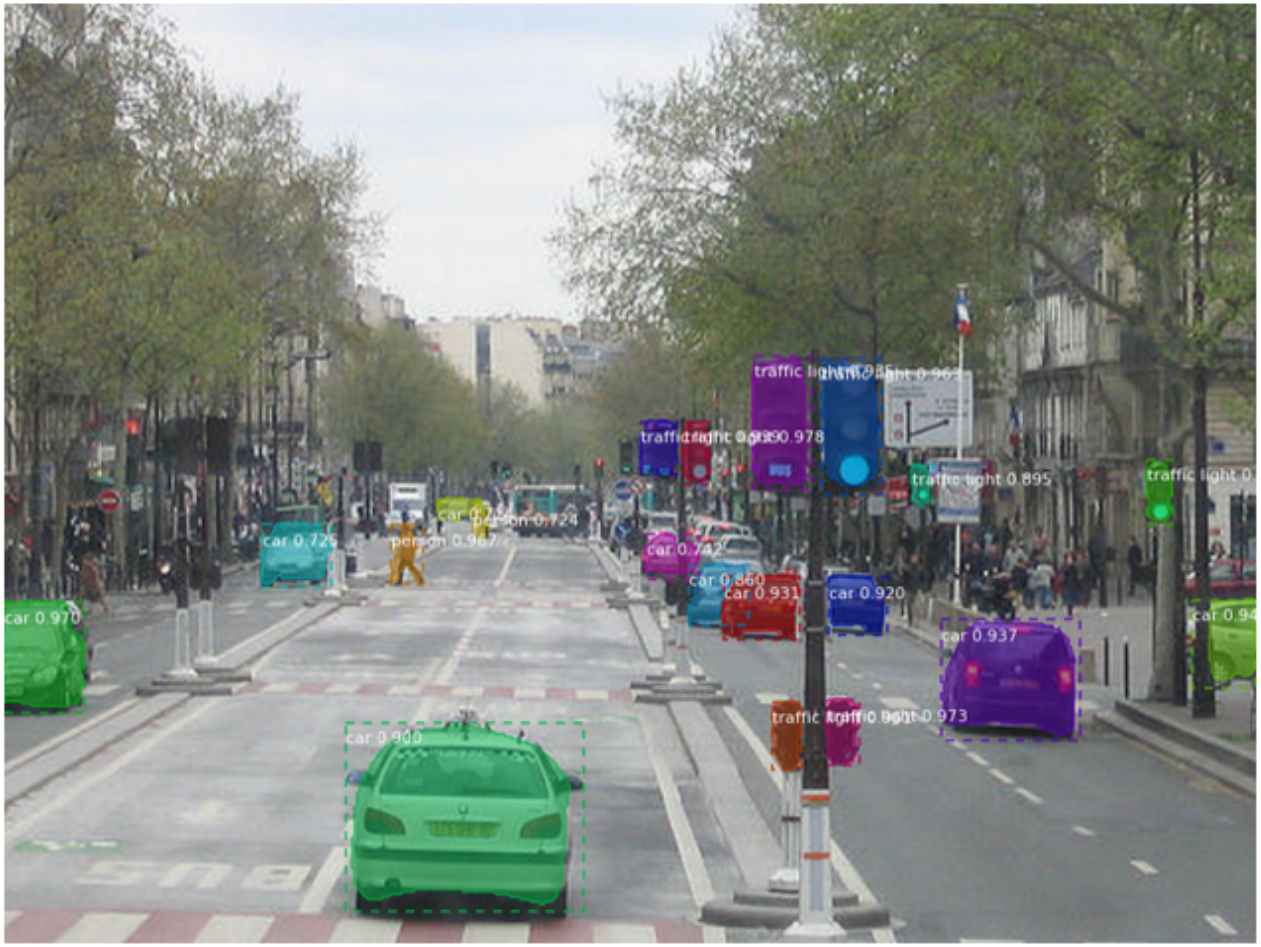
Instance segmentation

- **Instance segmentation** goes one step further: it predicts a class label and a precise **2D shape mask**.
- For each detected object m, the model predicts: class label $y_m \in \{1, \ldots, C\}$ and a binary mask $M_m \in \{0, 1\}^{H \times W}$.
- The mask tells us which pixels belong to object instance m:

$$M_m(i, j) = \begin{cases} 1, & \text{if pixel } (i, j) \text{ belongs to instance } m, \\ 0, & \text{otherwise.} \end{cases}$$

Example: object detection and instance segmentation

Identify each individual car separately, even if several cars appear in the image (Mask R-CNN, from this webstie)



Semantic segmentation

- **Semantic segmentation** predicts a class label for every pixel.
- For an image of height H and width W, the output is $\boldsymbol{Y} \in \{1, \ldots, C\}^{H \times W}$
- Equivalently, each pixel (i, j) receives a label $Y_{ij} \in \{1, \ldots, C\}$, e.g.

$$Y_{ij} = \begin{cases} \text{road}, & \text{if pixel } (i,j) \text{ belongs to road,} \\ \text{car}, & \text{if pixel } (i,j) \text{ belongs to a car,} \\ \text{person}, & \text{if pixel } (i,j) \text{ belongs to a person.} \end{cases}$$

- In contrast to **instance segmentation**, **semantic segmentation** groups all pixels of the same class together; it does **not** distinguish between separate objects of the same class.
- Example: if there are three cars, semantic segmentation labels all car pixels as `car`, but it does not say which pixels belong to car 1, car 2, or car 3. **Instance segmentation**, in contrast, separates the three cars into distinct object instances.

Panoptic segmentation

- **Panoptic segmentation** combines two segmentation tasks:
  1. **Semantic segmentation** for amorphous background regions, called **stuff** classes, e.g. sky, road, grass, water.
  2. **Instance segmentation** for countable objects, called **thing** classes, e.g. car, person, dog, bicycle.
- The output assigns every pixel both a semantic class label and, for thing classes, an instance identity.
- Mathematically, for each pixel (i, j), we predict $Y_{ij} = (c_{ij}, m_{ij})$, where $c_{ij} \in \{1, \ldots, C\}$ is the semantic class, and m_{ij} is the instance index.
- For **stuff** classes, the instance index is not needed: $m_{ij} = \varnothing$.
- For **thing** classes, m_{ij} separates different objects of the same class.
- Example: all road pixels share the class `road`, while three cars are labeled as separate car instances.

Encoder–decoder models for segmentation

- A common architecture for segmentation is an **encoder–decoder** network.
- The **encoder** maps the image to a lower-resolution representation with high-level features: $\boldsymbol{x} \in \mathbb{R}^{H \times W \times K} \longmapsto \boldsymbol{z} \in \mathbb{R}^{H' \times W' \times D}$, where usually $H' < H$ $W' < W$.
- The encoder reduces spatial resolution but increases the field of view.
- This can be done using: pooling, strided convolutions, dilated convolutions.
- The **decoder** upsamples the representation back to image resolution: $\boldsymbol{z} \in \mathbb{R}^{H' \times W' \times D} \longmapsto \hat{\boldsymbol{Y}} \in \{1, \ldots, C\}^{H \times W}$.
- The result is a dense prediction map: each pixel receives a predicted class label.
- For semantic segmentation, this is a class label per pixel.
- For panoptic segmentation, the output must also distinguish object instances for thing classes.

Illustration: encoder-decoder architecture

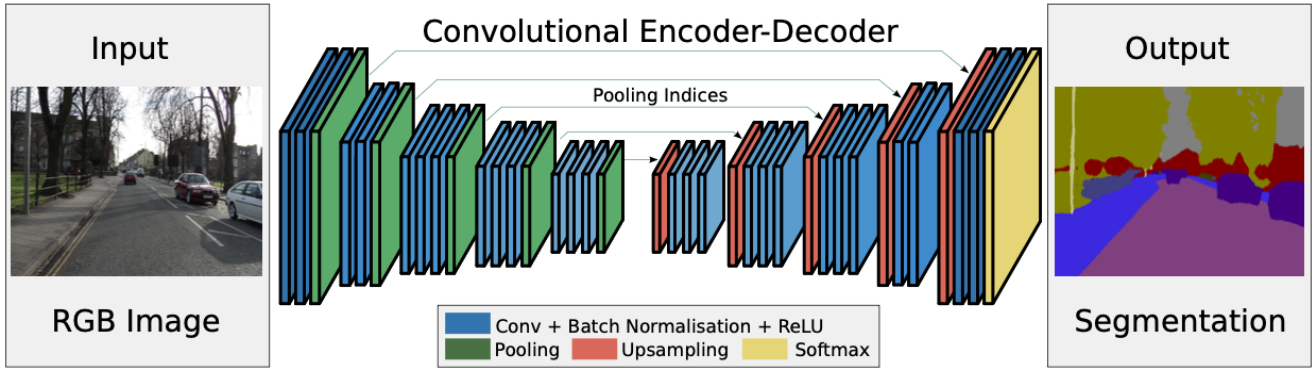


Illustration of an encoder-decoder CNN for semantic segmentation. The encoder uses convolution (which downsamples), and the decoder uses transposed convolution (which upsamples). Copied from PML: Chapter 14, Figure 14.29.

Human pose estimation

- We could train an object detector to find people and predict their 2D shape, for example a bounding box or segmentation mask.
- Alternatively, we can train the model to predict the locations of a **fixed set of skeletal keypoints**.
- This task is called **human pose estimation**.
- Examples of keypoints include: head, shoulders, elbows, hands, knees, and feet.
- For a fixed set of K keypoints, the output for one person can be written as $\boldsymbol{p} = (\boldsymbol{p}_1, \ldots, \boldsymbol{p}_K)$, where each 2D keypoint is $\boldsymbol{p}_k = (x_k, y_k) \in \mathbb{R}^2$.
- The predicted keypoints form a simple skeleton describing the person's pose.
- Examples of pose-estimation systems include **PersonLab** ([Pap+18]) and **OpenPose** ([Cao+18]); see [Bab19] for a review.
- Similar ideas can be extended to **3D properties** of detected objects, but the main limitation is data.
- Collecting labeled 3D training data is difficult because humans cannot easily annotate precise 3D positions from ordinary images.

Illustration: human pose estimation

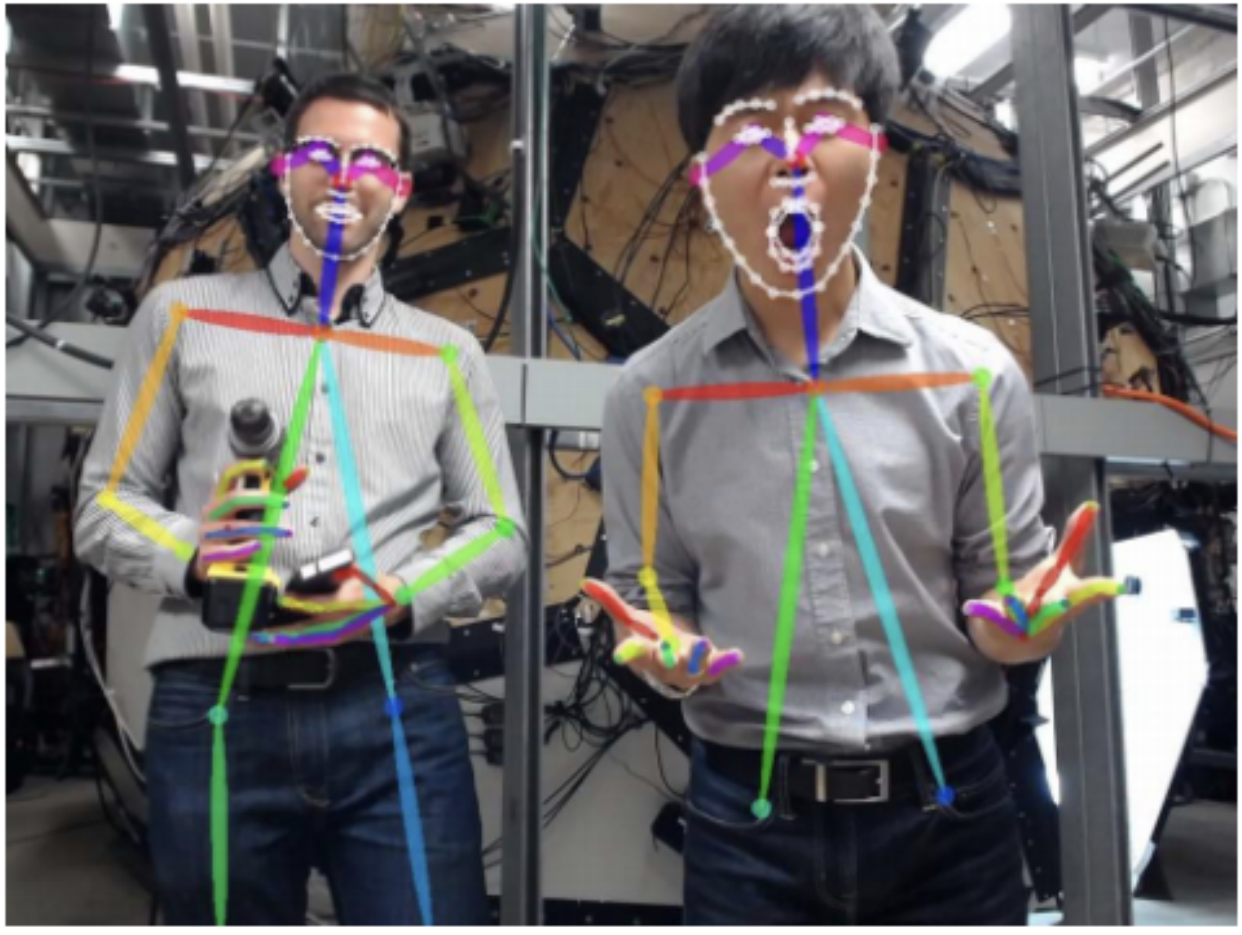


Illustration of keypoint detection for body, hands and face using the OpenPose system. Copied from PML: Chapter 14, Figure 14.32.

Neural Networks for Images

- Introduction
- Common layers
- Common architectures for image classification
- Solving other discriminative vision tasks with CNNs
- **Generating images by inverting CNNs**

From discriminative to generative

- A CNN trained for image classification is a **discriminative** model $p(y \mid \mathbf{x})$: takes an image, returns a distribution over C class labels
- In this section we "invert" this model, converting it into a (conditional) **generative image model** $p(\mathbf{x} \mid y)$ – it lets us generate images belonging to a specific class

Converting a classifier into a generative model

- Define a joint distribution $p(\mathbf{x}, y) = p(\mathbf{x})\, p(y \mid \mathbf{x})$, where $p(y \mid \mathbf{x})$ is the CNN classifier and $p(\mathbf{x})$ is some prior over images
- Conditioning $\mathbf{x}$ on a specific value of y (a specific class label) gives a conditional generative model: $p(\mathbf{x} \mid y) \propto p(\mathbf{x})\, p(y \mid \mathbf{x})$
- Caveat: a discriminative classifier was trained to **throw away** information, so $p(y \mid \mathbf{x})$ is not invertible – the prior $p(\mathbf{x})$ plays a crucial regularizing role

Sampling: Langevin-style updates

- We want to sample images that are both **plausible images** and **likely to belong to class** c.
- One option is **Metropolis–Hastings**, using the class-conditional objective $\mathcal{E}_c(\mathbf{x}) = \log p(y = c \mid \mathbf{x}) + \log p(\mathbf{x})$.
- Since this objective is differentiable with respect to the image $\mathbf{x}$, we can use gradient-informed proposals.
- In the **Metropolis-adjusted Langevin algorithm (MALA)**, we propose a new image from $q(\mathbf{x}' \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}(\mathbf{x}), \epsilon\mathbf{I})$, where $\boldsymbol{\mu}(\mathbf{x}) = \mathbf{x} + \frac{\epsilon}{2}\nabla_{\mathbf{x}}\mathcal{E}_c(\mathbf{x})$.
- Intuition: the proposal moves the current image slightly toward higher class-conditional probability, while still adding random noise.
- If we drop the rejection step and accept every proposal, we obtain so called the **unadjusted Langevin algorithm**.

Sampling: Langevin-style updates

- A more flexible variant scales the prior and likelihood gradients separately, giving an stochastic gradient descent (SGD)-like update **on the pixels** rather than on the model parameters: $\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon_1 \frac{\partial \log p(\mathbf{x}_t)}{\partial \mathbf{x}_t} + \epsilon_2 \frac{\partial \log p(y=c|\mathbf{x}_t)}{\partial \mathbf{x}_t} + \mathcal{N}(\mathbf{0}, \epsilon_3^2 \mathbf{I})$.
- The three terms have different roles:
  - ϵ_1 controls how strongly we move toward images that are plausible under the prior $p(\mathbf{x})$.
  - ϵ_2 controls how strongly we move toward images classified as class c.
  - ϵ_3 controls the amount of injected noise, which gives sample diversity.
- If $\epsilon_3 = 0$, the update becomes deterministic and approximately searches for the **most likely image** for class c rather than sampling diverse images.

Image priors

- Specifying only the class label is not enough information to pin down what kind of images we want – we also need a prior $p(\mathbf{x})$ over "plausible" images
- The choice of prior can have a large effect on the quality of the generated image

Gaussian prior

- Simplest choice: $p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \mathbf{0}, \mathbf{I})$ (assumes pixels are centered) – prevents pixels from taking extreme values
- Prior-term gradient: $\nabla_{\mathbf{x}} \log p(\mathbf{x}_t) = -\mathbf{x}_t$
- With $\epsilon_2 = 1,\ \epsilon_3 = 0$, the overall update simplifies to $\mathbf{x}_{t+1} = (1 - \epsilon_1)\mathbf{x}_t + \frac{\partial \log p(y=c|\mathbf{x}_t)}{\partial \mathbf{x}_t}$

goose

ostrich

Images that maximize the probability of ImageNet classes ''goose'' and ''ostrich'' under a simple Gaussian prior. From here.

Total variation (TV) prior

- A Gaussian prior alone gives unrealistic images; add regularization.
- **Total variation (TV) norm**: integral of the per-pixel gradients, approximated as
  $\mathrm{TV}(\mathbf{x}) = \sum_{ijk}(x_{ijk} - x_{i+1,j,k})^2 + (x_{ijk} - x_{i,j+1,k})^2$

where x_{ijk} is the pixel value at row i, column j, channel k

- One can rewrite TF in terms of the horizontal/vertical **Sobel edge detector** applied to each channel:
  $\mathrm{TV}(\mathbf{x}) = \sum_k \|\mathbf{H}(\mathbf{x}_{:,:,k})\|_F^2 + \|\mathbf{V}(\mathbf{x}_{:,:,k})\|_F^2$
- Using $p(\mathbf{x}) \propto \exp(-\mathrm{TV}(\mathbf{x}))$ discourages high-frequency artefacts

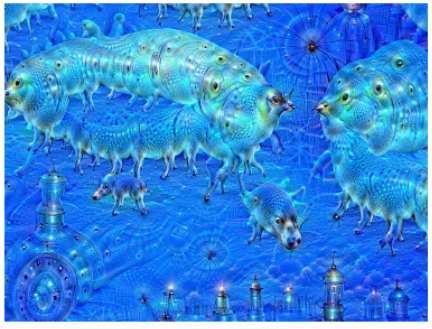
Anemone Fish | Banana | Parachute | Screw

DeepDream

- A more artistic application: generate versions of an input image that **emphasize** certain features
- View the pre-trained classifier as a feature extractor; amplify features from a chosen set of layers $\mathcal{L}$ via an energy/loss function $\mathcal{L}(\mathbf{x}) = \sum_{l \in \mathcal{L}} \bar{\phi}_l(\mathbf{x})$, where $\bar{\phi}_l$ is the (spatially averaged) feature vector for layer l
- Gradient ascent on this energy gives **DeepDream** ([MOT15]) – the model amplifies features only hinted at in the original image, generating more and more of them

DeepDream – Illustration

(a)



(b)

(c)

Example: starting from a jellyfish image and a CNN trained on ImageNet, iterating produces a hybrid of the input and dog-like "hallucinations" – unsurprising, since ImageNet has so many dog breeds in its label set. Copied from PML: Chapter 14, Figure 14.37.

Neural style transfer

Idea

- DeepDream is one way to use CNNs to make "art," but the results can look rather unsettling and offer little user control
- **Neural style transfer**: user specifies a **style image** $\mathbf{x}_s$ and a **content image** $\mathbf{x}_c$; the system generates a new image $\mathbf{x}$ that re-renders $\mathbf{x}_c$ in the style of $\mathbf{x}_s$

How does it work? $\rightarrow$ The energy function

Style transfer optimizes the energy function

$$\mathcal{L}(\mathbf{x} \mid \mathbf{x}_s, \mathbf{x}_c) = \lambda_{TV}\mathcal{L}_{TV}(\mathbf{x}) + \lambda_c\mathcal{L}_{\text{content}}(\mathbf{x}, \mathbf{x}_c) + \lambda_s\mathcal{L}_{\text{style}}(\mathbf{x}, \mathbf{x}_s)$$

- First term: the TV prior from before
- **Content loss**: compares feature maps of a pre-trained CNN $\phi(\mathbf{x})$ at a chosen "content layer" ℓ:

$$\mathcal{L}_{\text{content}}(\mathbf{x}, \mathbf{x}_c) = \frac{1}{C_\ell H_\ell W_\ell}\|\phi_\ell(\mathbf{x}) - \phi_\ell(\mathbf{x}_c)\|_2^2$$

Capturing style: the Gram matrix

- Visual **style** $\approx$ the statistical distribution of certain image features: location may not matter, but **co-occurrence** does
- Capture co-occurrence statistics at layer ℓ with the **Gram matrix**:

$$G_\ell(\mathbf{x})_{c,d} = \frac{1}{H_\ell W_\ell} \sum_{h=1}^{H_\ell} \sum_{w=1}^{W_\ell} \phi_\ell(\mathbf{x})_{h,w,c}\, \phi_\ell(\mathbf{x})_{h,w,d}$$

- $G_\ell(\mathbf{x})$ is $C_\ell \times C_\ell$, proportional to the uncentered covariance of the C_ℓ-dimensional feature vectors sampled across all $H_\ell W_\ell$ spatial locations
  - **Style loss** at layer ℓ: $\mathcal{L}^\ell_{\text{style}}(\mathbf{x}, \mathbf{x}_s) = \|\mathbf{G}_\ell(\mathbf{x}) - \mathbf{G}_\ell(\mathbf{x}_s)\|_F^2$
    - Overall style loss sums over a chosen set of layers $\mathcal{S}$: $\mathcal{L}_{\text{style}}(\mathbf{x}, \mathbf{x}_s) = \sum_{\ell \in \mathcal{S}} \mathcal{L}^\ell_{\text{style}}(\mathbf{x}, \mathbf{x}_s)$
    - Lower layers tend to capture visual texture; higher layers capture object layout

References

- ESLII: Chapter 11.3 - 11.5
- PML: Chapter 14
- PRML Chapter 5